SE444: Introduction to Artificial Intelligence
Mock Final Exam — Solutions — Fall 2025

Prof. Anis Koubaa
akoubaa@alfaisal.edu
Alfaisal University

Date: TBD, January 2026 Time: 120 minutes

Instructions

- This is a closed-book exam. The University's code of ethics applies.
- Answer each exam question to the best of your knowledge.
- Clearly state any assumptions made that might be needed to understand your solution.
- Show your work – any sign of serious effort will be considered.
- No marks will be given for not making any effort.

Student Information

Student ID:	Student Name:

Summary of Exam Questions

Part	Topic	Time	Score	Mark	CLOs
Part I MCQ	BN Structure / d-Separation	4 min	1		CLO 1
	Bayes' Rule Application	4 min	1		CLO 1
	Decision Theory / MDPs	4 min	1		CLO 1, 2
	Game Theory — Nash Equilibrium	4 min	1		CLO 1, 3
	Classical Planning (STRIPS)	4 min	1		CLO 1, 2
	Project Code — Search	4 min	1		CLO 4
Part II BN	BN Construction + d-Separation	12 min	4		CLO 1
	Bayesian Inference	12 min	4		CLO 2
	MEU Decision Making	9 min	3		CLO 2, 3
Part III MDP	MDP & Value Iteration	15 min	5		CLO 2
Part IV Plan/GT	STRIPS Planning	9 min	3		CLO 2
	Nash Equilibrium	9 min	3		CLO 3
Part V Code	Search Algorithm Analysis	6 min	3		CLO 4
	Bayesian Belief Updates	6 min	2		CLO 4
Total for Part I (Multiple Choice)		24 min	6		
Total for Part II (Bayesian Networks)		33 min	11		
Total for Part III (MDPs)		15 min	5		
Total for Part IV (Planning/Game Theory)		18 min	6		
Total for Part V (Code Analysis)		12 min	5		
Total Points		102 min	**30**		

Part I. Critical-Thinking Multiple Choice Questions

Total Score: 6 points **Total Time:** 24 minutes

Instructions: Each question may have **one or more correct answers**. Select **all that apply**. Partial credit will be awarded for partially correct answers.

Answer Key Summary

Option	Q1	Q2	Q3	Q4	Q5	Q6
A	**X**		**X**	**X**	**X**	**X**
B		**X**	**X**		**X**	
C		**X**	**X**	**X**		**X**
D	**X**	**X**	**X**	**X**	**X**	
E				**X**	**X**	**X**

Correct Answers:

- **Q1 (d-Separation):** A, D
- **Q2 (Bayes):** B, C, D
- **Q3 (MDPs):** A, B, C, D
- **Q4 (Game Theory):** A, C, D, E
- **Q5 (Planning):** A, B, D, E
- **Q6 (Code):** A, C, E

Question I.1: Bayesian Network Structure & d-Separation

Time: 4 min **Score:** 1 pt **CLOs:** CLO 1

Consider the Bayesian Network: $A \to C \leftarrow B$, $C \to D$ (where C is a collider).
Which conditional independence statements are TRUE?

A) $A \perp B$ (unconditionally)

B) $A \perp B \mid C$

C) $A \perp B \mid D$

D) $A \perp D \mid C$

E) $C \perp D$ (unconditionally)

Answer: A, D
Explanation:

A) **Correct.** A and B are unconditionally independent. The path $A \to C \leftarrow B$ is blocked because C is a collider that is not observed.

B) **Incorrect.** When C is observed, the collider becomes "open" and A and B become dependent (explaining away effect).

C) **Incorrect.** Observing D (a descendant of collider C) also opens the collider path, making A and B

dependent.

D) **Correct.** Given C, the path $A \to C \to D$ is blocked at C (chain is blocked when middle node is observed). So $A \perp D \mid C$.

E) **Incorrect.** $C \to D$ means C and D are directly connected and therefore dependent.

Key Concept: Colliders block paths when unobserved but open paths when observed (or when descendants are observed).

Question I.2: Bayes' Rule Application

Time: 4 min **Score:** 1 pt **CLOs:** CLO 1

A cybersecurity system: $P(\text{Intrusion}) = 0.01$, $P(\text{Alarm}|\text{Intrusion}) = 0.95$, $P(\text{Alarm}|\neg\text{Intrusion}) = 0.05$. The alarm triggers. Which are TRUE?

A) $P(\text{Intrusion}|\text{Alarm}) > 0.5$

B) $P(\text{Intrusion}|\text{Alarm}) < P(\text{Alarm}|\text{Intrusion})$

C) The base rate significantly affects the posterior

D) If $P(\text{Intrusion})$ increases to 0.10, then $P(\text{Intrusion}|\text{Alarm}) > 0.5$

E) Bayes' rule is unnecessary; $P(\text{Intrusion}|\text{Alarm}) = P(\text{Alarm}|\text{Intrusion})$

Answer: B, C, D

Calculation:

$$P(I|A) = \frac{P(A|I) \cdot P(I)}{P(A)}$$

$$\begin{aligned} P(A) &= P(A|I)P(I) + P(A|\neg I)P(\neg I) \\ &= 0.95 \times 0.01 + 0.05 \times 0.99 = 0.0095 + 0.0495 = 0.059 \end{aligned}$$

$$P(I|A) = \frac{0.95 \times 0.01}{0.059} = \frac{0.0095}{0.059} \approx 0.161$$

Analysis:

A) **Incorrect.** $P(I|A) \approx 0.161 < 0.5$

B) **Correct.** $P(I|A) \approx 0.161 < 0.95 = P(A|I)$

C) **Correct.** The low base rate (1%) significantly reduces the posterior despite high sensor accuracy.

D) **Correct.** With $P(I) = 0.10$: $P(A) = 0.95(0.1) + 0.05(0.9) = 0.14$, $P(I|A) = 0.095/0.14 \approx 0.68 > 0.5$

E) **Incorrect.** Bayes' rule is essential; $P(I|A) \neq P(A|I)$.

Key Concept: Base rate neglect — even with accurate tests, rare events have low posterior probability.

Question I.3: Decision Theory & MDPs

Time: 4 min **Score:** 1 pt **CLOs:** CLO 1, 2

In an MDP with discount factor $\gamma = 0.9$, which statements are TRUE?

A) If $\gamma = 0$, the agent only considers immediate rewards

B) Bellman equation $V(s) = \max_a[R(s,a) + \gamma \sum_{s'} P(s'|s,a)V(s')]$ converges for $0 \leq \gamma < 1$

C) Value iteration and policy iteration produce the same optimal policy

D) Higher γ (closer to 1) makes the agent more "farsighted"

E) In deterministic MDPs, optimal policy is the same regardless of γ

Answer: A, B, C, D

A) **Correct.** $\gamma = 0$ means $V(s) = \max_a R(s, a)$ — only immediate rewards matter.

B) **Correct.** Value iteration converges for $\gamma < 1$ (contraction mapping theorem).

C) **Correct.** Both algorithms find the same optimal policy (same fixed point).

D) **Correct.** Higher γ gives more weight to future rewards, making agent "farsighted."

E) **Incorrect.** In deterministic MDPs, different γ can change optimal policy. Consider: short path with small reward vs. long path with large reward.

Question I.4: Game Theory — Nash Equilibrium

Time: 4 min **Score:** 1 pt **CLOs:** CLO 1, 3

Consider the game (payoffs Row, Col):

	Left	**Right**
Up	(3, 3)	(1, 4)
Down	(4, 1)	(2, 2)

Which are TRUE?

A) Both players have a strictly dominant strategy

B) (Up, Left) is a Nash Equilibrium

C) (Down, Right) is a Nash Equilibrium

D) "Down" strictly dominates "Up" for Row player

E) The Nash Equilibrium yields lower payoffs for both players than (Up, Left)

Answer: A, C, D, E
Analysis:

A) **Correct.** Row's dominant strategy is Down (4¿3 vs Left, 2¿1 vs Right). Column's dominant strategy is Right (4¿3 vs Up, 2¿1 vs Down). Both players have strictly dominant strategies.

B) **Incorrect.** At (Up,Left): Row can improve by switching to Down (4¿3). Not a NE.

C) **Correct.** At (Down,Right): Row gets 2, switching to Up gives 1. Column gets 2, switching to Left gives 1. Neither can improve unilaterally. This is a NE.

D) **Correct.** Down **strictly dominates** Up for Row player:

- Against Left: Down(4) > Up(3)
- Against Right: Down(2) > Up(1)

Since Down yields a strictly higher payoff than Up in *every* column, Down strictly dominates Up.

E) **Correct.** The NE (Down,Right)=(2,2) gives both players 2. The outcome (Up,Left)=(3,3) gives both players 3. So the NE yields lower payoffs for both.

Key Concept: When both players follow their dominant strategies, they reach the Nash Equilibrium (Down,Right)=(2,2), even though (Up,Left)=(3,3) would be better for both. This illustrates the tension between individual rationality and collective welfare.

Question I.5: Classical Planning — STRIPS

Time: 4 min **Score:** 1 pt **CLOs:** CLO 1, 2

Consider the following STRIPS action schema:

```
Action: Load(cargo, truck, loc)
  Precond: At(cargo, loc), At(truck, loc), ¬In(cargo, truck)
  Add:     In(cargo, truck)
  Delete:  At(cargo, loc)
```

Which statements are TRUE?

A) Forward search starts from initial state and applies actions until goal

B) Closed World Assumption means unmentioned atoms are false

C) After Load(C1,T1,Depot), At(C1,Depot) is still true

D) Backward search can be more efficient when goal is specific

E) STRIPS assumes deterministic, fully observable effects

Answer: A, B, D, E

A) **Correct.** Forward search (progression) starts from initial state and applies applicable actions.

B) **Correct.** CWA: any ground atom not explicitly true is assumed false.

C) **Incorrect.** At(C1,Depot) is in the Delete List, so it becomes false after Load.

D) **Correct.** Backward search can be more efficient when goal is specific and initial state is large.

E) **Correct.** STRIPS assumes deterministic effects and full observability of state.

Lecture References (Lecture 9: Automated Planning):

(A) Forward Search (Progression): Slides 35–40; Topic 6

(B) Closed World Assumption: Slides 15–18; Topic 2

(C) Delete List / STRIPS Effects: Slides 22–28; Topic 3

(D) Backward Search (Regression): Slides 41–48; Topic 7

(E) STRIPS Assumptions: Slides 10–14; Topic 1

Question I.6: Project Code — Search Algorithms

Time: 4 min **Score:** 1 pt **CLOs:** CLO 4

```
frontier = [(0, counter, start)]
while frontier:
    cost, _, current = heapq.heappop(frontier)
    if current in explored: continue
    explored.add(current)
    if current == goal: return path, cost
    for neighbor in env.get_neighbors(current):
        new_cost = cost + env.get_cost(current, neighbor)
        heapq.heappush(frontier, (new_cost, counter, neighbor))
```

Which are TRUE?

A) This implements Uniform Cost Search (UCS)

B) This implements A* Search with a heuristic

C) Goal test happens when node is **expanded** (popped)

D) With equal edge costs, this behaves like BFS

E) `counter` breaks ties in the priority queue

Answer: A, C, E

A) **Correct.** The code uses *cost* (g-value only) as priority, no heuristic. This is UCS.

B) **Incorrect.** No heuristic function $h()$ is used in the priority.

C) **Correct.** Goal test happens after `heappop` (when expanded), not when generated.

D) **Marked Incorrect (but debatable).** With equal costs, UCS uses a priority queue while BFS uses a FIFO queue — different data structures. However, **behaviorally** they are equivalent:

- With equal step costs, UCS expands nodes in non-decreasing depth order
- The `counter` provides FIFO tie-breaking among equal-cost nodes
- This produces the *same expansion order* as BFS

If "behaves like" means *same expansion order and result*, D could be considered TRUE. We mark it FALSE based on the *implementation* distinction.

E) **Correct.** The counter breaks ties for nodes with same cost (FIFO order among equals).

Part II. Probability & Bayesian Networks

Total Score: 11 points **Total Time:** 33 minutes

Question II.1: BN Construction & d-Separation (4 pts, 12 min)

Scenario: A company monitors network security with variables:

- `NetworkTraffic (N)`: High/Normal volume
- `IntrusionAttempt (I)`: Actual intrusion occurring
- `SystemLoad (S)`: Server under heavy load
- `FirewallAlert (F)`: Firewall generates alert
- `LogAnomaly (L)`: System logs show anomalies

Dependencies: I and S both cause high N. F depends on N and I. L depends on I and S.
Tasks:

(a) [2 pts] Draw the Bayesian Network structure.

(b) [1 pt] Are $I \perp S \mid F$? Use d-separation.

(c) [1 pt] Which node in this network is a **collider**? What happens when this collider node is observed?

(a) Bayesian Network Structure [2 pts]:

Lecture Reference: BN structure and construction are covered in Lecture 11, Part 1 (Slides 15–30).

How to Build a Bayesian Network:
A Bayesian Network (BN) is a directed acyclic graph (DAG) where:

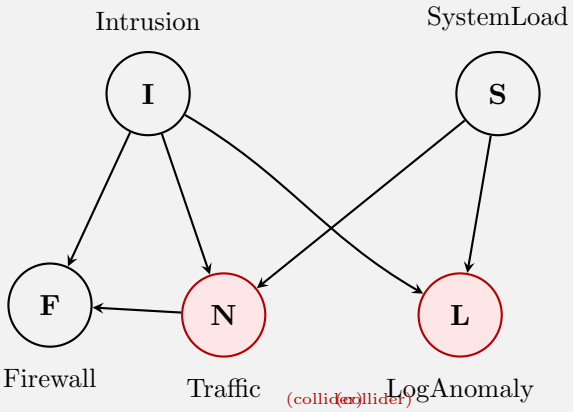
- **Nodes** represent random variables
- **Edges** represent direct causal/probabilistic dependencies (arrow points from cause to effect)
- A node is **conditionally independent** of its non-descendants given its parents

---

Applying to the Network Security Scenario:
From the problem statement, we identify the causal relationships:

1. "I and S both cause high N" $\Rightarrow$ Draw arrows $I \to N$ and $S \to N$
2. "F depends on N and I" $\Rightarrow$ Draw arrows $N \to F$ and $I \to F$
3. "L depends on I and S" $\Rightarrow$ Draw arrows $I \to L$ and $S \to L$

The Resulting Network:



Summary of Edges:

Edge	From → To	Meaning
$I \to N$	Intrusion → Traffic	Intrusion causes high network traffic
$S \to N$	SystemLoad → Traffic	Heavy server load causes high traffic
$I \to F$	Intrusion → Firewall	Intrusion triggers firewall alert
$N \to F$	Traffic → Firewall	High traffic triggers firewall alert
$I \to L$	Intrusion → LogAnomaly	Intrusion creates log anomalies
$S \to L$	SystemLoad → LogAnomaly	Heavy load creates log anomalies

Key Observation: Nodes N and L are **colliders** (nodes with multiple parents). This is important for d-separation analysis.

Grading: 1pt for correct arrows, 1pt for identifying collider structure.

(b) Is $I \perp S \mid F$? d-Separation analysis [1 pt]:

Lecture Reference: d-Separation rules are covered in Lecture 11, Part 1 (Slides 45–60).

What is d-Separation?
d-Separation is a graphical criterion to determine if two variables are conditionally independent given some evidence. The key rules are:

1. **Chain** ($A \to B \to C$): Observing B blocks the path
2. **Fork** ($A \leftarrow B \to C$): Observing B blocks the path
3. **Collider** ($A \to B \leftarrow C$): Path is **blocked by default**, but **opens when B or any descendant of B is observed**

Applying d-Separation to our question: Is $I \perp S \mid F$?
Step 1: Find all paths between I and S
There are two paths:

- Path 1: $I \to N \leftarrow S$ (through collider N)
- Path 2: $I \to L \leftarrow S$ (through collider L)

Step 2: Check if each path is blocked given F
Path 1: $I \to N \leftarrow S$

- N is a collider, so this path is normally **blocked**
- But wait! F is a **descendant of** N (since $N \to F$)

- When we observe F, we gain information about N
- Therefore, observing F **opens** this collider path!
- **Result: Path 1 is ACTIVE**

Path 2: $I \rightarrow L \leftarrow S$

- L is a collider, so this path is **blocked**
- F is NOT a descendant of L
- **Result: Path 2 is BLOCKED**

Step 3: Conclusion
Since Path 1 is active, there is an open path between I and S when conditioning on F.

Answer: NO, I and S are NOT conditionally independent given F

Intuition: If we observe a firewall alert (F), and we know there was no intrusion, we become more suspicious that heavy system load caused the alert (through high traffic). The firewall alert creates a dependency between intrusion and system load!

Grading: 1pt for correct answer with path tracing justification.

(c) Collider and What Happens When It's Observed [1 pt]:

Lecture Reference: Colliders and Explaining Away are covered in Lecture 11, Part 1 (Slides 55–65).

What is a Collider?
A **collider** is a node that has two or more parent nodes pointing into it. In our network:

- N (NetworkTraffic) is a collider: $I \rightarrow N \leftarrow S$
- L (LogAnomaly) is also a collider: $I \rightarrow L \leftarrow S$

What Happens When a Collider is Observed?
When we observe a collider node, something interesting happens: **the parent variables become dependent**, even though they were originally independent. This is called the **"Explaining Away"** effect.

Example using N (NetworkTraffic):

1. **Before observing N:** Intrusion (I) and SystemLoad (S) are independent—knowing one tells us nothing about the other.
2. **After observing $N = \text{High}$:** Now I and S become dependent! Why?
   - High traffic must have a cause: either an intrusion OR heavy system load (or both)
   - If we learn $S = \text{True}$ (server is under load), this **explains** the high traffic
   - This makes intrusion **less likely**—we have an alternative explanation!

Mathematical Expression:

$$P(I = T \mid N = \text{High}, S = T) < P(I = T \mid N = \text{High})$$

Real-world Intuition: Imagine you see your network traffic spike. You think, "Is this an attack or just our servers working hard?" If someone tells you the servers are running a big batch job ($S = T$), you breathe easier—the traffic is probably just normal server activity, not a hacker.

Key Takeaway: Observing a collider "opens" the path between its parents, creating a dependency where none existed before.

Grading: 0.5pt for identifying collider, 0.5pt for explaining what happens when observed.

Question II.2: Bayesian Inference (4 pts, 12 min)

Scenario: A smart home system uses a motion sensor to automatically control lights. When someone is **Home**, the **Motion** sensor is more likely to detect movement. When **Motion** is detected, the **Lights** turn on automatically.

Bayesian Network: Home → Motion → Lights

$P(\text{Home} = T)$	0.6

$P(\text{Motion}\|\text{Home})$	Home= T	Home= F
Motion= T	0.9	0.1

$P(\text{Lights}\|\text{Motion})$	Motion= T	Motion= F
Lights= T	0.95	0.05

Tasks:

(a) [2 pts] Given that the lights are ON, compute $P(\text{Home} = T \mid \text{Lights} = T)$ using enumeration.

(b) [2 pts] Apply Variable Elimination to compute $P(\text{Home} = T \mid \text{Lights} = T)$. Show factor tables.

(a) Compute $P(H = T \mid L = T)$ by enumeration [2 pts]:

Step 1: Apply Bayes' Rule

$$P(H = T|L = T) = \frac{P(L = T|H = T) \cdot P(H = T)}{P(L = T)}$$

Step 2: Compute $P(L = T|H = T)$ by marginalization over M

$$\begin{aligned} P(L = T|H = T) &= \sum_m P(L = T|M = m) \cdot P(M = m|H = T) \\ &= P(L = T|M = T) \cdot P(M = T|H = T) + P(L = T|M = F) \cdot P(M = F|H = T) \\ &= 0.95 \times 0.9 + 0.05 \times 0.1 \\ &= 0.855 + 0.005 = 0.86 \end{aligned}$$

Step 3: Compute $P(L = T|H = F)$

$$\begin{aligned} P(L = T|H = F) &= 0.95 \times 0.1 + 0.05 \times 0.9 \\ &= 0.095 + 0.045 = 0.14 \end{aligned}$$

Step 4: Compute $P(L = T)$ (total probability)

$$\begin{aligned} P(L = T) &= P(L = T|H = T) \cdot P(H = T) + P(L = T|H = F) \cdot P(H = F) \\ &= 0.86 \times 0.6 + 0.14 \times 0.4 \\ &= 0.516 + 0.056 = 0.572 \end{aligned}$$

Step 5: Final Answer

$$P(H = T|L = T) = \frac{0.86 \times 0.6}{0.572} = \frac{0.516}{0.572} \approx \boxed{0.902}$$

Grading: 1pt for correct marginalization, 1pt for correct final answer.

(b) Variable Elimination to compute $P(\textbf{Home} = T \mid \textbf{Lights} = T)$ [2 pts]:

Lecture Reference: Variable Elimination is covered in Lecture 11, Part 2 (Slides 25–40).

Algorithm Overview:
Variable Elimination (VE) is an efficient algorithm for exact inference in Bayesian Networks. Instead of computing the full joint distribution, VE works with smaller tables called **factors**.

1. **Create initial factors** from the CPTs (Conditional Probability Tables)
2. **Set evidence** by restricting factors to observed values
3. **Eliminate hidden variables** one at a time: multiply relevant factors, then sum out the variable
4. **Multiply remaining factors** and **normalize** to get the final probability

---

Applying VE to our Smart Home problem:
Query: $P(\text{Home} \mid \text{Lights} = T)$
Evidence: Lights $= T$ (observed)
Hidden variable to eliminate: Motion

Step 1: Create Initial Factors
We create one factor for each CPT in our network:

- $f_1(\text{Home}) = P(\text{Home})$: Prior probability of being home
- $f_2(\text{Motion}, \text{Home}) = P(\text{Motion}|\text{Home})$: Motion sensor given home status
- $f_3(\text{Lights}, \text{Motion}) = P(\text{Lights}|\text{Motion})$: Lights given motion

Step 2: Set Evidence (Lights $= T$)
We observe that Lights are ON. Restrict f_3 to only the rows where Lights$= T$:

$$f_3'(\text{Motion}) = P(\text{Lights} = T \mid \text{Motion})$$

Motion	$f_3'(\text{Motion})$	**Meaning**
T	0.95	If motion detected, 95% chance lights are ON
F	0.05	If no motion, only 5% chance lights are ON

Step 3: Eliminate Motion
Multiply $f_2 \times f_3'$, then sum out Motion to get a new factor over Home only:

$$f_4(\text{Home}) = \sum_{\text{Motion}} f_2(\text{Motion}, \text{Home}) \cdot f_3'(\text{Motion})$$

Home	**Motion**	f_2 (Motion|Home)	f_3' (Lights=T|Motion)	**Product**
T	T	0.9	0.95	$0.9 \times 0.95 = 0.855$
T	F	0.1	0.05	$0.1 \times 0.05 = 0.005$
F	T	0.1	0.95	$0.1 \times 0.95 = 0.095$
F	F	0.9	0.05	$0.9 \times 0.05 = 0.045$

Sum out Motion (add rows with same Home value):

$$f_4(\text{Home}) = \sum_{\text{Motion}} f_2(\text{Motion}, \text{Home}) \cdot f_3'(\text{Motion})$$

- $f_4(\text{Home} = T) = \sum_M f_2(M, T) \cdot f_3'(M) = \sum_M P(M|\text{Home} = T) \cdot P(\text{Lights} = T|M) = 0.855 + 0.005 = 0.86$
- $f_4(\text{Home} = F) = \sum_M f_2(M, F) \cdot f_3'(M) = \sum_M P(M|\text{Home} = F) \cdot P(\text{Lights} = T|M) = 0.095 + 0.045 = 0.14$

Intuition: This factor tells us: "Given lights are ON, how likely is each Home status?" (before considering the prior)

Step 4: Multiply with Prior and Normalize
Multiply f_4 with the prior f_1(Home):

Home	f_1 (Prior)	f_4 (From Step 3)	$f_5 = f_1 \times f_4$
T	0.6	0.86	$0.6 \times 0.86 = 0.516$
F	0.4	0.14	$0.4 \times 0.14 = 0.056$

Normalize (divide by sum to get probabilities):

$$P(\text{Home} = T \mid \text{Lights} = T) = \frac{0.516}{0.516 + 0.056} = \frac{0.516}{0.572} \approx \boxed{0.902}$$

Interpretation: If the lights are ON, there is a 90.2% probability that someone is home.

Grading: 1pt for correct factor operations, 1pt for correct normalization.

Question II.3: Decision Under Uncertainty (3 pts, 9 min)

Scenario: A drone delivery company must decide how to deliver a package. The drone's success and reward depend on the weather conditions, which are uncertain.

Weather Probabilities: Clear (0.6), Cloudy (0.3), Storm (0.1)

Action	**Clear**	**Cloudy**	**Storm**
Fly Direct	+100	+60	−50
Take Detour	+70	+70	+30
Wait	+50	+50	+80

Tasks:

(a) [2 pts] Compute the **Expected Utility (EU)** for each action using: $EU(a) = \sum_s P(s) \cdot U(a, s)$

(b) [1 pt] Which action is optimal according to the **Maximum Expected Utility (MEU)** principle? Justify.

(a) Expected Utility (EU) for each action [2 pts]:

Lecture Reference: Decision Theory and Expected Utility are covered in Lecture 12 (Slides 15–30).

What is Expected Utility?
Expected Utility (EU) is the weighted average of utilities across all possible outcomes, where weights are the probabilities of each outcome:

$$\text{Expected Utility}(a) = EU(a) = \sum_s P(s) \cdot U(a, s)$$

where $P(s)$ is the probability of state s, and $U(a, s)$ is the utility of taking action a in state s.

Calculating EU for each action:
Fly Direct:

$$\begin{aligned} EU(\text{Fly Direct}) &= P(\text{Clear}) \cdot U(\text{Fly}, \text{Clear}) + P(\text{Cloudy}) \cdot U(\text{Fly}, \text{Cloudy}) + P(\text{Storm}) \cdot U(\text{Fly}, \text{Storm}) \\ &= 0.6 \times 100 + 0.3 \times 60 + 0.1 \times (-50) \\ &= 60 + 18 - 5 = \boxed{73} \end{aligned}$$

Take Detour:

$$EU(\text{Take Detour}) = 0.6 \times 70 + 0.3 \times 70 + 0.1 \times 30$$
$$= 42 + 21 + 3 = \boxed{66}$$

Wait:

$$EU(\text{Wait}) = 0.6 \times 50 + 0.3 \times 50 + 0.1 \times 80$$
$$= 30 + 15 + 8 = \boxed{53}$$

Summary:

Action	Expected Utility
Fly Direct	73
Take Detour	66
Wait	53

Grading: Each correct EU calculation is worth 0.67pt.

(b) Maximum Expected Utility (MEU) optimal action [1 pt]:

What is Maximum Expected Utility (MEU)?
The MEU principle states that a rational agent should choose the action that maximizes expected utility:

$$a^* = \arg\max_a EU(a)$$

Applying MEU to our problem:
From our calculations:

- $EU(\text{Fly Direct}) = 73$
- $EU(\text{Take Detour}) = 66$
- $EU(\text{Wait}) = 53$

Since $73 > 66 > 53$:

$$\boxed{\text{MEU-optimal action: Fly Direct with } EU = 73}$$

Intuition: While Fly Direct has high risk in storms (-50 utility), the low probability of storms (10%) combined with high rewards in good weather makes it the best choice *in expectation*. A rational agent should not be overly cautious about unlikely bad outcomes if the expected payoff is higher.
Grading: 1pt for correct answer with justification.

Part III. MDPs & Value Iteration

Total Score: 5 points **Total Time:** 15 minutes

Question III.1: MDP & Value Iteration (5 pts, 15 min)

Scenario: A warehouse robot needs to navigate a 2×3 grid to reach a goal position. The robot receives a reward when it reaches the goal, but incurs a small cost for each step taken.

Grid Layout:

S1	S2	S3
S4	S5	**G** (+10)

Parameters:

- **Goal reward:** $R(G) = +10$
- **Living reward (step cost):** -1 per step
- **Discount factor:** $\gamma = 0.9$
- **Transitions:** 80% intended direction, 10% slip left, 10% slip right. Walls keep robot in place.

Tasks:

(a) [1 pt] Write the MDP components (S, A, P, R, γ).

(b) [2 pts] Perform 2 iterations of Value Iteration for S3 and S5. Start with $V_0 = 0$. Use: $V_{k+1}(s) = \max_a[R(s,a) + \gamma \sum_{s'} P(s'|s,a)V_k(s')]$. Note: $V(G) = 10$.

(c) [2 pts] Extract optimal policy $\pi^*(s)$ for S3 and S5. Justify.

(a) MDP Components [1 pt]:

Lecture Reference: MDP fundamentals are covered in Lecture 13 (Slides 10–25).

What is an MDP?
A Markov Decision Process (MDP) is a mathematical framework for sequential decision-making under uncertainty. An MDP is defined by 5 components:

1. **States (S):** All possible situations the agent can be in
2. **Actions (A):** Choices available to the agent
3. **Transition function P(s'|s,a):** Probability of reaching state s' when taking action a in state s
4. **Reward function R(s,a):** Immediate reward/cost for taking action a in state s
5. **Discount factor (γ):** How much future rewards are discounted ($0 \leq \gamma \leq 1$)

Applying to the Warehouse Robot:

- $S = \{S1, S2, S3, S4, S5, G\}$ (6 states, where G is the terminal/goal state)
- $A = \{\text{Up}, \text{Down}, \text{Left}, \text{Right}\}$ (4 possible actions)
- $P(s'|s,a)$: Stochastic transition function
  - 0.8 probability: move in intended direction
  - 0.1 probability: slip to left of intended direction
  - 0.1 probability: slip to right of intended direction
  - If hitting a wall, stay in current state

- $R(s, a) = -1$ for all non-terminal states (living penalty encourages reaching goal quickly)
- $R(G) = +10$ (reward for reaching the goal)
- $\gamma = 0.9$ (future rewards are slightly discounted)

Grading: 0.2pt for each component correctly specified.

(b) Value Iteration — 2 iterations for S3 and S5 [2 pts]:

Lecture Reference: Value Iteration is covered in Lecture 13 (Slides 35–55).

What is Value Iteration?
Value Iteration is an algorithm to find the optimal value function $V^*(s)$ for each state. It repeatedly applies the **Bellman Update** until convergence:

$$V_{k+1}(s) = \max_a \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) V_k(s') \right]$$

Intuition: The value of a state equals the best immediate reward plus the discounted expected value of where we end up.

Setup:

- **Initial values:** $V_0(s) = 0$ for all non-terminal states
- **Goal value:** $V(G) = 10$ (fixed, since G is terminal)
- **Grid reference:**

S1	S2	S3
S4	S5	G

Slip convention:

- Action Down ⇒ slip-left = Left, slip-right = Right
- Action Right ⇒ slip-left = Up, slip-right = Down

=== Iteration 1 ===

For S3 (directly above G):
Best action is **Down** (toward G):

- 0.8 probability → G (intended)
- 0.1 probability → S2 (slip left)
- 0.1 probability → wall = stay at S3 (slip right)

$$\begin{aligned} V_1(S3) &= R(S3, \text{Down}) + \gamma \sum_{s'} P(s'|S3, \text{Down}) \cdot V_0(s') \\ &= -1 + 0.9 \times [0.8 \times V(G) + 0.1 \times V_0(S2) + 0.1 \times V_0(S3)] \\ &= -1 + 0.9 \times [0.8 \times 10 + 0.1 \times 0 + 0.1 \times 0] \\ &= -1 + 0.9 \times 8 = -1 + 7.2 = \boxed{6.2} \end{aligned}$$

For S5 (directly left of G):
Best action is **Right** (toward G):

- 0.8 probability → G (intended)
- 0.1 probability → S2 (slip left = Up)
- 0.1 probability → wall = stay at S5 (slip right = Down)

$$\begin{aligned} V_1(S5) &= R(S5, \text{Right}) + \gamma \sum_{s'} P(s'|S5, \text{Right}) \cdot V_0(s') \\ &= -1 + 0.9 \times [0.8 \times 10 + 0.1 \times 0 + 0.1 \times 0] \\ &= -1 + 0.9 \times 8 = -1 + 7.2 = \boxed{6.2} \end{aligned}$$

For other states (S1, S2, S4): Cannot reach G in one step, so $V_1 = -1 + 0.9 \times 0 = -1$.

=== Iteration 2 ===
Now we use V_1 values: $V_1(S2) = -1$, $V_1(S3) = 6.2$, $V_1(S5) = 6.2$

For S3:

$$\begin{aligned} V_2(S3) &= -1 + 0.9 \times [0.8 \times V(G) + 0.1 \times V_1(S2) + 0.1 \times V_1(S3)] \\ &= -1 + 0.9 \times [0.8 \times 10 + 0.1 \times (-1) + 0.1 \times 6.2] \\ &= -1 + 0.9 \times [8 - 0.1 + 0.62] \\ &= -1 + 0.9 \times 8.52 = -1 + 7.668 = \boxed{6.67} \end{aligned}$$

For S5:

$$\begin{aligned} V_2(S5) &= -1 + 0.9 \times [0.8 \times V(G) + 0.1 \times V_1(S2) + 0.1 \times V_1(S5)] \\ &= -1 + 0.9 \times [8 + 0.1 \times (-1) + 0.1 \times 6.2] \\ &= -1 + 0.9 \times 8.52 = -1 + 7.668 = \boxed{6.67} \end{aligned}$$

Summary Table:

State	V_0	V_1	V_2
S3	0	6.2	6.67
S5	0	6.2	6.67

Observation: Values are increasing and converging. Both S3 and S5 have the same value because they are both one step away from G with the same transition probabilities.
Grading: 1pt per state for correct iteration calculations.

(c) Optimal Policy Extraction [2 pts]:

Lecture Reference: Policy extraction is covered in Lecture 13 (Slides 55–60).

What is Policy Extraction?
After Value Iteration converges, we extract the optimal policy by choosing the action that maximizes the Bellman equation at each state:

$$\pi^*(s) = \arg\max_a \left[R(s,a) + \gamma \sum_{s'} P(s'|s,a) V^*(s') \right]$$

Intuition: Pick the action that leads to the highest expected value.

Optimal Policy:

State	$\pi^*(s)$	Reasoning
S3	$\downarrow$ **Down**	Moves directly toward goal G (80% success)
S5	$\rightarrow$ **Right**	Moves directly toward goal G (80% success)

Justification:
The optimal policy directs the robot toward the goal G via the shortest path because:

1. **Discount factor ($\gamma = 0.9$):** Future rewards are worth less than immediate rewards, so reaching G quickly is preferred.
2. **Living penalty (-1/step):** Each step costs the robot, so minimizing the number of steps is important.
3. **Stochastic transitions (80/10/10):** Aiming directly at G gives the highest probability (80%) of reaching it in one step. Any other action would require more steps on average.

Visual Policy:

$\rightarrow$	$\rightarrow$	$\downarrow$
$\rightarrow$	$\rightarrow$	**G**

Grading: 1pt for correct policy, 1pt for justification.

Part IV. Planning & Game Theory

Total Score: 6 points **Total Time:** 18 minutes

Question IV.1: Classical Planning — STRIPS (3 pts, 9 min)

E-Commerce Fulfillment: Predicates: `At(pkg,loc)`, `In(pkg,truck)`, `TruckAt(truck,dock)`, `Sorted(pkg)`.
Initial: At(P1,Intake), At(P2,Intake), TruckAt(T1,DockA), ¬Sorted(P1), ¬Sorted(P2)
Goal: In(P1,T1), In(P2,T1)
Tasks:

(a) [1 pt] Define STRIPS action `Load(pkg,truck,dock)`: Preconditions, Add List, Delete List.

(b) [2 pts] Show forward search plan to achieve goal. For each step: action applied, key state changes.

(a) STRIPS Action `Load(pkg, truck, dock)` [1 pt]:
Preconditions:

- `At(pkg, dock)` — Package must be at the loading dock
- `TruckAt(truck, dock)` — Truck must be at the same dock
- `Sorted(pkg)` — Package must have been sorted first
- ¬`In(pkg, truck)` — Package not already in truck

Add List (Effects that become true):

- `In(pkg, truck)` — Package is now in the truck

Delete List (Effects that become false):

- `At(pkg, dock)` — Package is no longer at the dock

Grading: 0.5pt for correct preconditions, 0.25pt for add list, 0.25pt for delete list.

(b) Forward Search Plan [2 pts]:
Initial State:

```
At(P1, Intake), At(P2, Intake), TruckAt(T1, DockA),
~Sorted(P1), ~Sorted(P2), ~In(P1, T1), ~In(P2, T1)
```

Goal State: `In(P1, T1), In(P2, T1)`

Step 1: Sort(P1, Intake)

- Precond: `At(P1, Intake)` — satisfied
- Add: `Sorted(P1)`
- State: `At(P1,Intake), At(P2,Intake), TruckAt(T1,DockA), Sorted(P1)`

Step 2: Move(P1, Intake, DockA)

- Precond: `At(P1, Intake)` — satisfied
- Add: `At(P1, DockA)`
- Delete: `At(P1, Intake)`
- State: `At(P1,DockA), At(P2,Intake), TruckAt(T1,DockA), Sorted(P1)`

Step 3: Load(P1, T1, DockA)

- Precond: `At(P1,DockA), TruckAt(T1,DockA), Sorted(P1)` — all satisfied
- Add: `In(P1, T1)`
- Delete: `At(P1, DockA)`
- State: `In(P1,T1), At(P2,Intake), TruckAt(T1,DockA), Sorted(P1)`

Step 4: Sort(P2, Intake)

- Add: `Sorted(P2)`

Step 5: Move(P2, Intake, DockA)

- Add: `At(P2, DockA)`
- Delete: `At(P2, Intake)`

Step 6: Load(P2, T1, DockA)

- Add: `In(P2, T1)`
- Delete: `At(P2, DockA)`
- **Final State:** `In(P1,T1), In(P2,T1)` — **GOAL ACHIEVED ✓**

Plan: Sort(P1) → Move(P1) → Load(P1) → Sort(P2) → Move(P2) → Load(P2)
Grading: 1pt for correct sequence, 1pt for showing state changes.

Question IV.2: Game Theory — Nash Equilibrium (3 pts, 9 min)

Tech Startup Competition: Two startups choose pricing: Premium, Budget, or Freemium.

A \ B	Premium	Budget	Freemium
Premium	(8, 8)	(3, 10)	(4, 9)
Budget	(10, 3)	(5, 5)	(6, 4)
Freemium	(9, 4)	(4, 6)	(7, 7)

Tasks:

(a) [1 pt] Identify any strictly dominated strategies. Apply IESDS if applicable.

(b) [2 pts] Find all pure-strategy Nash Equilibria. Mark best responses for each player.

(a) Dominated Strategies / IESDS [1 pt]:
Check for Player A (Row):

- Premium vs Budget: Premium gives (8,3,4), Budget gives (10,5,6). Budget > Premium in all cases. **Premium is dominated by Budget for A.**
- Budget vs Freemium: Budget (10,5,6), Freemium (9,4,7). Neither dominates the other.

Check for Player B (Column):

- Premium vs Budget: Premium gives (8,3,4), Budget gives (10,5,6). Budget > Premium in all cases (from B's perspective: 8,3,4 vs 10,5,6). Wait, let me check B's payoffs properly.
- B's payoffs: Premium column (8,3,4), Budget column (10,5,6), Freemium column (9,4,7).
- Comparing columns for B: Budget (10,5,6) vs Premium (8,3,4) — Budget dominates Premium for B too.

After eliminating Premium for both players:
Reduced game:

A \ B	Budget	Freemium
Budget	(5, 5)	(6, 4)
Freemium	(4, 6)	(7, 7)

No further domination in reduced game.
Grading: 1pt for correctly identifying dominated strategies.

(b) Pure-Strategy Nash Equilibria [2 pts]:
Method: Find best responses for each player.
Player A's Best Response (underline A's best payoff in each column):

- If B plays Premium: A's payoffs are 8, 10, 9. Best: Budget (10)
- If B plays Budget: A's payoffs are 3, 5, 4. Best: Budget (5)
- If B plays Freemium: A's payoffs are 4, 6, 7. Best: Freemium (7)

Player B's Best Response (underline B's best payoff in each row):

- If A plays Premium: B's payoffs are 8, 10, 9. Best: Budget (10)
- If A plays Budget: B's payoffs are 3, 5, 4. Best: Budget (5)
- If A plays Freemium: B's payoffs are 4, 6, 7. Best: Freemium (7)

Finding Nash Equilibria (cells where both are playing best response):

A \ B	Premium	Budget	Freemium
Premium	(8, 8)	(3, 10)	(4, 9)
Budget	(10, 3)	(5, 5)	(6, 4)
Freemium	(9, 4)	(4, 6)	(7, 7)

Nash Equilibria:

1. **(Budget, Budget) = (5, 5)** — Both underlined
2. **(Freemium, Freemium) = (7, 7)** — Both underlined

Grading: 1pt for correct best response analysis, 1pt for identifying both NE.

Part V. Project Code Analysis

Total Score: 5 points **Total Time:** 12 minutes

Question V.1: Search Algorithm Analysis (3 pts, 6 min)

Given: A* search code snippet from the project.

```
def astar(env, start, goal, heuristic='manhattan'):
    h = lambda pos: env.manhattan_distance(pos, goal)
    g_score = {start: 0}
    f_score = {start: h(start)}
    frontier = [(f_score[start], counter, start)]

    while frontier:
        current_f, _, current = heapq.heappop(frontier)
        if current in explored: continue
        explored.add(current)
        if current == goal: return path, g_score[current]
        for neighbor in env.get_neighbors(current):
            tentative_g = g_score[current] + env.get_cost(current, neighbor)
            if neighbor not in g_score or tentative_g < g_score[neighbor]:
                g_score[neighbor] = tentative_g
                f_score[neighbor] = tentative_g + h(neighbor)
                heapq.heappush(frontier, (f_score[neighbor], counter, neighbor))
```

Tasks:

(a) [1 pt] What do `g_score`, `f_score`, and `h()` represent? Explain $f(n) = g(n) + h(n)$.

(b) [1 pt] Why check `if current in explored: continue`? What if removed?

(c) [1 pt] Is Manhattan distance admissible for 4-connected grid? Why/why not?

(a) g_score, f_score, h() meaning & $f(n) = g(n) + h(n)$ [1 pt]:

- **g_score[n]**: The actual cost from the start node to node n. This is the sum of edge costs along the path from start to n.
- **h(n)**: The heuristic function — an estimate of the cost from node n to the goal. In this code, it uses Manhattan distance: $|x_n - x_{goal}| + |y_n - y_{goal}|$.
- **f_score[n]**: The estimated total cost of the cheapest path through n: $f(n) = g(n) + h(n)$.

Relationship: $f(n) = g(n) + h(n)$

- $g(n)$ = actual cost from start to n (backward-looking)
- $h(n)$ = estimated cost from n to goal (forward-looking)
- $f(n)$ = estimated total path cost through n

A* expands nodes in order of f-value, which balances actual cost incurred with estimated cost remaining.
Grading: Full credit for explaining all three components and the relationship.

(b) Purpose of "if current in explored: continue" [1 pt]:
Purpose: This check prevents re-expanding nodes that have already been expanded.
Why it's needed:

1. A node can be added to the frontier multiple times with different g-scores if multiple paths lead to it.
2. When we pop a node, we may have already found a better path and expanded it previously.
3. This check skips redundant expansions of the same node.

What happens if removed:

- The algorithm becomes less efficient — nodes may be expanded multiple times.
- With an admissible heuristic, the algorithm still finds the optimal solution, but wastes computation.
- In the worst case, this can lead to exponential blowup in nodes expanded.

Grading: 0.5pt for purpose, 0.5pt for consequence of removal.

(c) Is Manhattan distance admissible for 4-connected grid? [1 pt]:
Answer: Yes, Manhattan distance is admissible for a 4-connected grid.
Proof of admissibility:
A heuristic h is admissible if $h(n) \leq h^*(n)$ for all nodes n, where $h^*(n)$ is the true optimal cost to the goal.
For a 4-connected grid (movement only Up/Down/Left/Right with uniform cost 1):

- Manhattan distance: $h(n) = |x_n - x_g| + |y_n - y_g|$
- This counts the minimum number of horizontal + vertical moves needed
- In a 4-connected grid without obstacles, this IS the actual shortest path
- With obstacles, the actual path may be longer, so $h(n) \leq h^*(n)$

Key insight: Manhattan distance assumes we can move directly (no obstacles), which gives a lower bound on the true cost. Since it never overestimates, it is admissible.
Grading: 1pt for correct answer with valid justification.

Question V.2: Bayesian Belief Updates (2 pts, 6 min)

Given: Belief update code with prior, sensor_reading, accuracy.

```
def update_single_cell_belief(prior, sensor_reading, sensor_accuracy=0.9):
    if sensor_reading:  # Obstacle detected
        likelihood_h = sensor_accuracy       # P(detect | obstacle)
        likelihood_not_h = 1 - sensor_accuracy  # P(detect | free)
    else:  # Clear
        likelihood_h = 1 - sensor_accuracy   # P(clear | obstacle)
        likelihood_not_h = sensor_accuracy   # P(clear | free)
    posterior, _ = bayes_update_full(prior, likelihood_h, likelihood_not_h)
    return posterior
```

Tasks:

(a) [1 pt] If prior=0.3, sensor_reading=True, accuracy=0.9, compute posterior. Show Bayes' rule.

(b) [1 pt] What happens if accuracy=0.5? Explain intuitively.

(a) Compute posterior (prior=0.3, detect=True, accuracy=0.9) [1 pt]:

Given:

- Prior: $P(\text{obstacle}) = 0.3$
- Sensor reading: True (obstacle detected)
- Sensor accuracy: 0.9

From the code:

- $P(\text{detect}|\text{obstacle}) = \text{sensor_accuracy} = 0.9$ (likelihood_h)
- $P(\text{detect}|\text{free}) = 1 - \text{sensor_accuracy} = 0.1$ (likelihood_not_h)

Apply Bayes' Rule:

$$P(\text{obstacle}|\text{detect}) = \frac{P(\text{detect}|\text{obstacle}) \cdot P(\text{obstacle})}{P(\text{detect})}$$

$$\begin{aligned} P(\text{detect}) &= P(\text{detect}|\text{obs}) \cdot P(\text{obs}) + P(\text{detect}|\text{free}) \cdot P(\text{free}) \\ &= 0.9 \times 0.3 + 0.1 \times 0.7 \\ &= 0.27 + 0.07 = 0.34 \end{aligned}$$

$$P(\text{obstacle}|\text{detect}) = \frac{0.9 \times 0.3}{0.34} = \frac{0.27}{0.34} \approx \boxed{0.794}$$

Interpretation: After receiving a positive detection, our belief in an obstacle increases from 30% to about 79.4%.

Grading: 1pt for correct calculation with steps shown.

(b) Effect when accuracy=0.5 [1 pt]:

Calculation with accuracy=0.5:

- $P(\text{detect}|\text{obstacle}) = 0.5$
- $P(\text{detect}|\text{free}) = 0.5$

$$P(\text{detect}) = 0.5 \times 0.3 + 0.5 \times 0.7 = 0.15 + 0.35 = 0.5$$

$$P(\text{obstacle}|\text{detect}) = \frac{0.5 \times 0.3}{0.5} = \frac{0.15}{0.5} = \boxed{0.3}$$

Result: The posterior equals the prior (0.3).

Intuitive Explanation:

When sensor accuracy is 0.5 (50%), the sensor is essentially a **random coin flip** — it gives "detect" with equal probability regardless of whether there's an obstacle or not. This means:

- The sensor provides **no information** about the actual state
- Observing a detection tells us nothing we didn't already know
- Therefore, our belief doesn't change: $P(\text{obstacle}|\text{detect}) = P(\text{obstacle})$

Key insight: A sensor with 50% accuracy is informationally useless — it's equivalent to not having a sensor at all.

Grading: 0.5pt for showing posterior = prior, 0.5pt for intuitive explanation.

— END OF SOLUTIONS —